

MINI-PROJET THÉORIE DES LANGAGES ET COMPILATION

Année 2025-2026

- GRAMMAIRE FORMELLE du Langage GLSimpleSQL

- Filière: LST GL S5
- Professeur: N. Mouhni

- Réalisé par :

MERIZAK FERDAOUSSE

KASRI CHOUAYB

HAJIR SALAH EDDINE

KHYAR IBTISSAM

➤ Introduction

- **1.1 Objet du Document**

Ce document présente la grammaire formelle complète du langage GLSimpleSQL, un sous-ensemble de SQL simplifié pour l'interpréteur développé dans le cadre du mini-projet.

- **1.2 Notation Utilisée**

Nous utilisons la notation EBNF (Extended Backus-Naur Form) :

- **::=** : Définition d'une production

- | : Alternative (ou)
- [] : Élément optionnel
- { } : Répétition (0 ou plusieurs fois)
- () : Groupement
- **Terminaux** : en guillemets "SELECT"
- **Non-terminaux** : en angles <requete>

• 1.3 Conventions

- Les mots-clés SQL en MAJUSCULES
- Les éléments lexicaux en minuscules
- Point-virgule ; obligatoire en fin de commande

➤ Structure Générale du Langage

• 2.1 Programme Principal

```

<programme>      ::=      {      <commande>      ";"      }

<commande>      ::=      <creation_table>
                        |
                        |
                        |
                        |
                        | <suppression_table>

```

• 2.2 Éléments Communs

```

<nom_table>      ::=      <identificateur>
<nom_champ>      ::=      <identificateur>

```

```

<identificateur> ::= <lettre> { <lettre> | <chiffre> | "_" }

<valeur> ::=
|
|
| <booléen>
| <chaîne>
| <entier>
| <reel>

```

➤ CREATE TABLE

- 3.1 Syntaxe Complète

```

<creation_table> ::= "CREATE" "TABLE" <nom_table>
                  "(" <definitions_champs> ")"

<definitions_champs> ::=
{ <definition_champ> "," <definition_champ> }

<definition_champ> ::= <nom_champ> <type_donnees>

<type_donnees> ::=
| "INT"
| "FLOAT"
| "VARCHAR"
| "BOOL"

```

➤ INSERT INTO

- 4.1 Syntaxe Complète

```

<insertion> ::= "INSERT" "INTO" <nom_table>
              "VALUES" "(" <liste_valeurs> ")"

<liste_valeurs> ::= <valeur> { "," <valeur> }

```

➤ SELECT

- 5.1 Syntaxe Complète

```

<selection> ::= "SELECT" <liste_selection>
              "FROM" <nom_table>
              [ "WHERE" <condition> ]

<liste_selection> ::= "*"
                  | <nom_champ> { "," <nom_champ> }

```

➤ UPDATE

- 6.1 Syntaxe Complète

```

<mise_a_jour> ::= "UPDATE" <nom_table>
                 "SET" <liste_assignations>
                 "WHERE" <condition>

<liste_assignations> ::= <assignation> { "," <assignation> }
<assignation> ::= <nom_champ> "=" <valeur>

```

➤ DELETE

- 7.1 Syntaxe Complète

```
<suppression> ::= "DELETE" "FROM" <nom_table>  
                [ "WHERE" <condition> ]
```

➤ DROP TABLE

- 8.1 Syntaxe Complète

```
<suppression_table> ::= "DROP" "TABLE" <nom_table>
```

➤ Conditions (WHERE)

- 9.1 Syntaxe des Conditions

```
<condition> ::= <expression> <opérateur_comparaison> <expression>  
                | <condition> "AND" <condition>  
                | <condition> "OR" <condition>  
  
<expression> ::= <nom_champ>  
                | <valeur>  
  
<opérateur_comparaison> ::= "="  
                | "!="  
                | "<"  
                | ">"  
                | "<="  
                | ">="
```

➤ Éléments Lexicaux

- 10.1 Types de Données

```
<entier> ::= [ "-" ] <chiffre> { <chiffre> }  
<reel> ::= [ "-" ] <chiffre> { <chiffre> } "." <chiffre> { <chiffre> }  
<chaine> ::= " " { <caractere> } "  
<booléen> ::= "TRUE" | "FALSE"
```

- 10.2 Caractères de Base

```
<lettre> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"  
<chiffre> ::= "0" | "1" | ... | "9"  
<caractere> ::= /* Tout caractère Unicode sauf délimiteurs */
```

➤ Règles Sémantiques

- 11.1 Contraintes Sémantiques

1. **Noms uniques** : Les noms de tables doivent être uniques
2. **Champs uniques** : Les noms de champs doivent être uniques dans une table
3. **Correspondance** : Le nombre de valeurs dans INSERT doit correspondre au nombre de champs de la table
4. **Tables existantes** : Les tables référencées doivent avoir été créées
5. **Champs existants** : Les champs référencés doivent exister dans la table

➤ Correspondance avec l'Implémentation

- **12.1 Lien avec le Code Bison**

Cette grammaire correspond directement à l'analyseur syntaxique :

- `<creation_table>` → règle `create_table` dans `parser.y`
- `<selection>` → règle `select_stmt` dans `parser.y`
- `<condition>` → règle `condition` dans `parser.y`

- **12.2 Évolutions Possibles**

- Ajout de `VARCHAR(n)` avec gestion de taille
- Ajout de `NOT` et parenthèses dans les conditions
- Support de `INSERT` avec liste de champs explicite